Lab Assignment 4

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Ryan Kapoor

Roll No. 1024150188

Batch: 2O23

1)

Create a class named 'Rectangle' with two data members – *length* and *breadth* and a function to calculate the area which is *length*breadth*. The class has three constructors which are:

(a) having no parameter – values of both length and breadth are assigned zero.
(b) having two numbers as parameters – the two numbers are assigned as length and breadth respectively.
(c) having one number as parameter – both length and breadth are assigned that number.

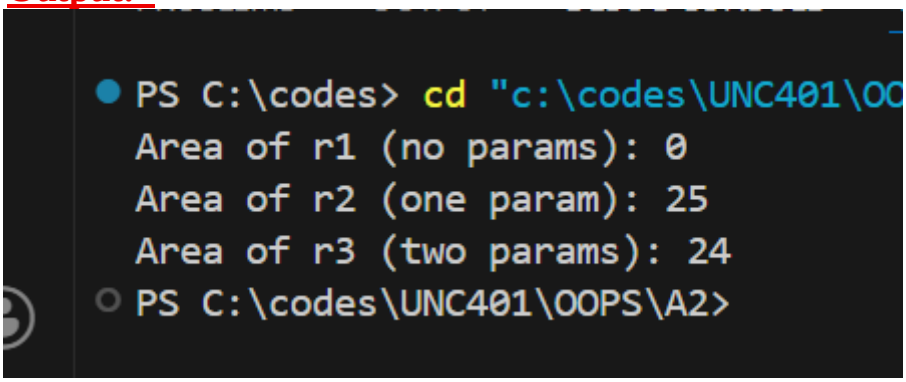
Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.

Input: -

```
#include <iostream>
using namespace std;
class Rectangle {
float length, breadth;
public:
// (a) Default constructor — no parameters
Rectangle() {
length = 0;
breadth = 0;
}
// (b) Two-parameter constructor
Rectangle(float l, float b) {
length = l;
breadth = b;
}
// (c) One-parameter constructor — both set to same value
Rectangle(float side) {
length = side;
breadth = side;
}
float area() {
return length * breadth;
}
};
int main() {
Rectangle r1; // calls (a): length=0, breadth=0
Rectangle r2(5.0f); // calls (c): length=5, breadth=5
Rectangle r3(4.0f, 6.0f); // calls (b): length=4, breadth=6
cout << "Area of r1 (no params): " << r1.area() << endl;
cout << "Area of r2 (one param): " << r2.area() << endl;
cout << "Area of r3 (two params): " << r3.area() << endl;
return 0;
}
/* Output:
```

*/}

Output: -


PS C:\codes> cd "c:\codes\UNC401\OO
Area of r1 (no params): 0
Area of r2 (one param): 25
Area of r3 (two params): 24
PS C:\codes\UNC401\OOPS\A2>

2) Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

```
#include <iostream>
using namespace std;
class Rectangle {
float length, breadth;
int id;
static int count;
public:

Rectangle() {
length = 0; breadth = 0;
id = ++count;
cout << "Constructor called for object " << id
<< " (default)" << endl;

}

Rectangle(float l, float b) {
length = l; breadth = b;
id = ++count;
cout << "Constructor called for object " << id
<< " (l=" << l << ", b=" << b << ")" << endl;

}

Rectangle(float side) {
length = side; breadth = side;
id = ++count;
cout << "Constructor called for object " << id
<< " (square side=" << side << ")" << endl;

}
float area() {
return length * breadth;
}

~Rectangle() {
cout << "Destructor called for object " << id << endl;
}
};
int Rectangle::count = 0;
int main() {
Rectangle arr[3];
Rectangle r1(7.0f);
Rectangle r2(3.0f, 9.0f);
cout << "\nAreas:" << endl;
for (int i = 0; i < 3; i++)
cout << " arr[" << i << "] area = " << arr[i].area() << endl;
cout << " r1 area = " << r1.area() << endl;
cout << " r2 area = " << r2.area() << endl;
```

```
cout << "\n-- Leaving main, destructors fire --" << endl;
return 0;
}
```

```
PS C:\codes> cd "c:\codes\UNC401\OOPS\A

-- Leaving main, destructors fire --
Destructor called for object 5
Destructor called for object 4
Destructor called for object 3
Destructor called for object 2
Destructor called for object 1
PS C:\codes\UNC401\OOPS\A2>
```

3)

Verify the following about destructor **by writing the program:**

(i) Name should begin with tilde sign (~) and must match class name.
(ii) There cannot be more than one destructor in a class.
(iii) Destructors do not allow any parameter.
(iv) They do not have any return type, just like constructors.

When you do not specify any destructor in a class, compiler generates a default destructor and inserts it into your code.

Answers :-

i) True

ii) True

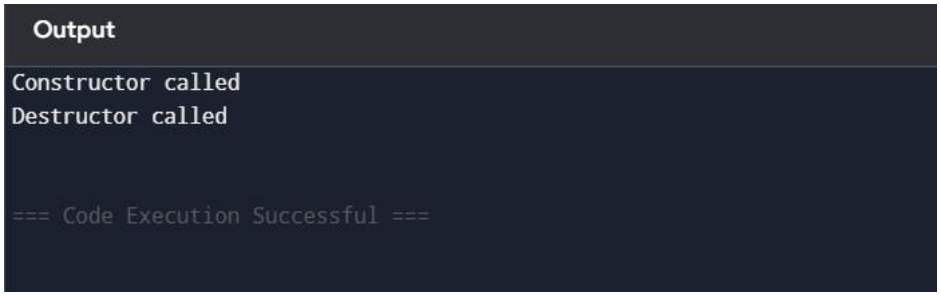
iii) True

iv) True

```cpp
#include <iostream>
using namespace std;
class Rectangle{
    public:
    int length, breadth;

    Rectangle(){
        cout<< "Constructor is called"<< endl;
    };
    ~Rectangle(){
        cout<< "Destructor is called"<< endl;

    };
};
int main(){
    Rectangle();

}
```

```
Constructor is called
Destructor is called
PS C:\Users\Harshita\Desktop\OOPS\Assignments\A4>
```

```cpp
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() {
        cout << "Constructor called\n";
    }

    ~Demo() {
        cout << "Destructor called\n";
    }
};

int main() {
    Demo obj;
    return 0;
}
```

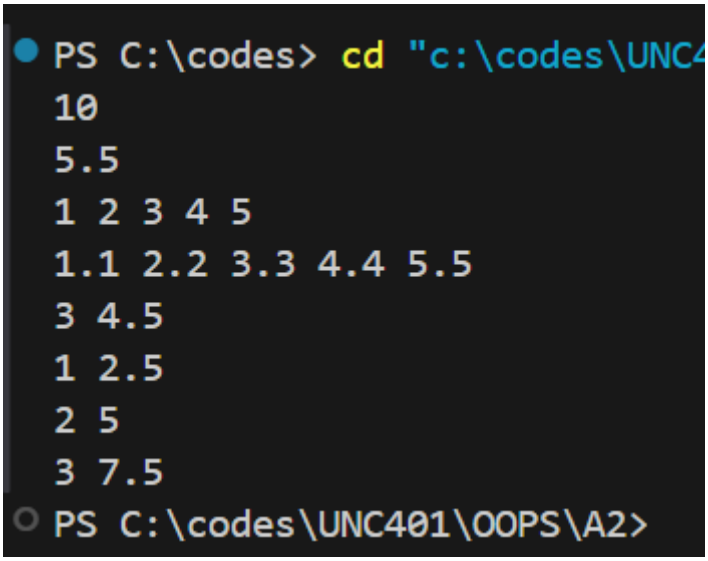


4)

Implement dynamic memory allocation. Use new and delete keywords. (For integer variable, float variable, integer array, float array, class objects, Array of Objects)

```
#include <iostream>
using namespace std;

class Sample {
public:
   int x;
   float y;

   void setData(int a, float b) {
      x = a;
      y = b;
   }

   void display() {
      cout << x << " " << y << endl;
   }
};

int main() {

   int *i = new int;
   *i = 10;
   cout << *i << endl;
   delete i;

   float *f = new float;
   *f = 5.5;
   cout << *f << endl;
   delete f;

   int n = 5;
   int *arr1 = new int[n];
   for(int j = 0; j < n; j++) {
      arr1[j] = j + 1;
      cout << arr1[j] << " ";
   }
   cout << endl;
   delete[] arr1;
```

```
    float *arr2 = new float[n];
    for(int j = 0; j < n; j++) {
        arr2[j] = (j + 1) * 1.1;
        cout << arr2[j] << " ";
    }
    cout << endl;
    delete[] arr2;

    Sample *obj = new Sample;
    obj->setData(3, 4.5);
    obj->display();
    delete obj;

    int m = 3;
    Sample *objArr = new Sample[m];
    for(int j = 0; j < m; j++) {
        objArr[j].setData(j+1, (j+1)*2.5);
    }
    for(int j = 0; j < m; j++) {
        objArr[j].display();
    }
    delete[] objArr;

    return 0;
}
```


PS C:\codes> cd "c:\codes\UNC
10
5.5
1 2 3 4 5
1.1 2.2 3.3 4.4 5.5
3 4.5
1 2.5
2 5
3 7.5
PS C:\codes\UNC401\OOPS\A2>